

Packwerk

What is it and how do we use it?

Steve Reinke & Christoph Wagner - webit! Gesellschaft für neue Medien mbH

Agenda

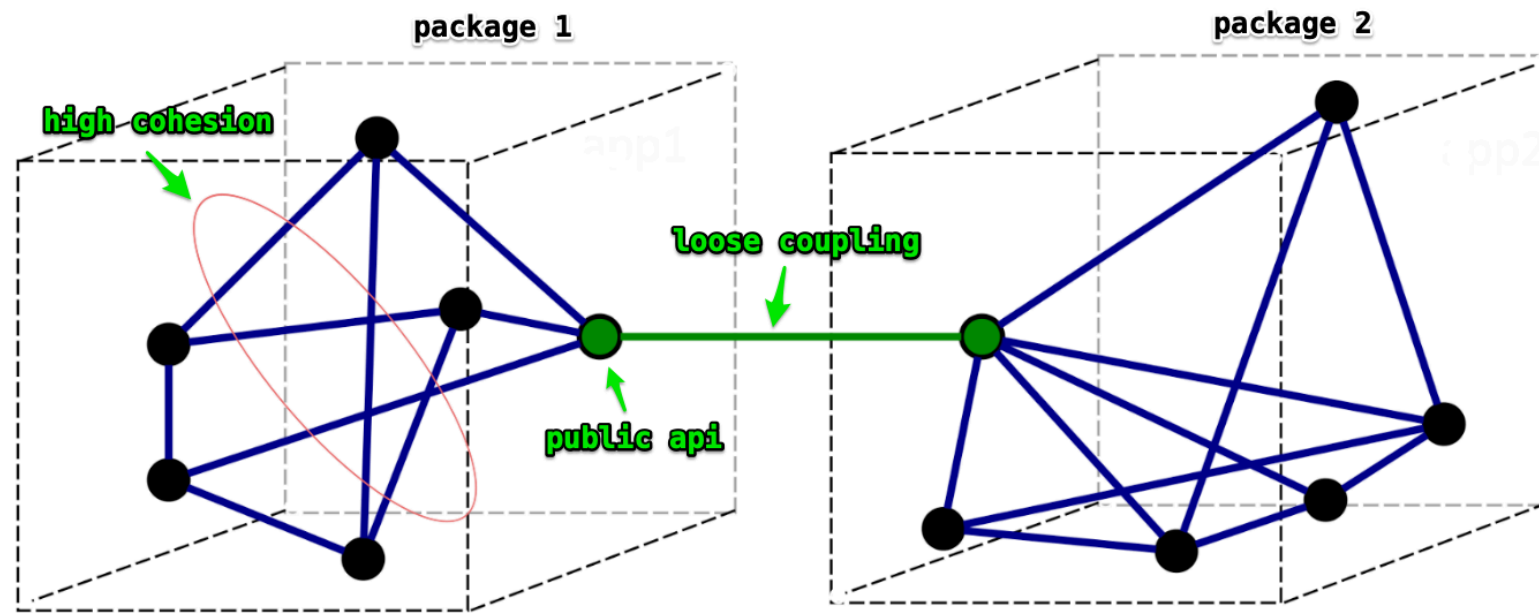
1. Motivation
2. Packwerk
3. Usage at webit!
4. Conclusion



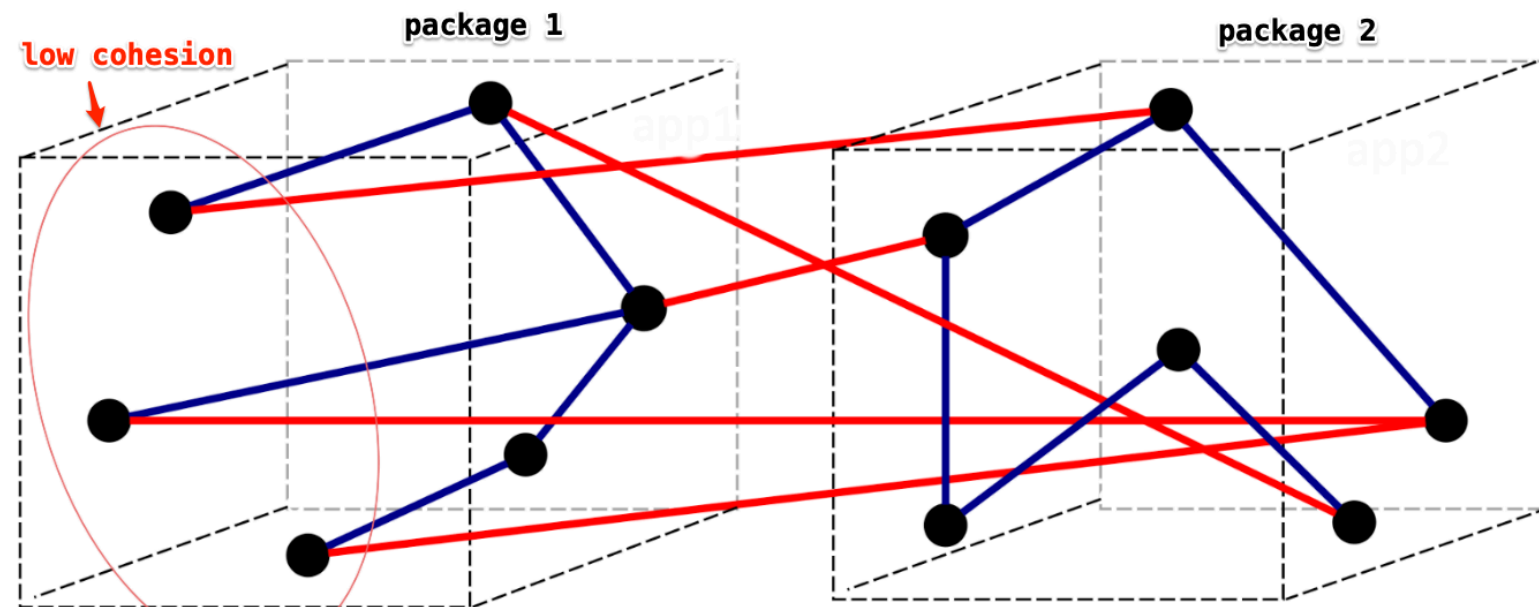
Motivation

The pain of a growing monolith

Have you ever made a code change that suddenly broke things you had nothing to do with?



a) Good



b) Bad

Rails provides structure – but not enough

Rails gives you a structure for files.

Who is allowed to talk to whom – nobody enforces that.

MVC and Convention over Configuration don't solve the problem of dependencies.

Why not just rely on discipline?

- People forget rules.
- New developers don't know them.
- Time pressure always wins.

Code reviews scale worse than automated rules.

→ How do you make dependencies **visible** — and rules enforceable?



Packwerk

What is it, how does it work?

Where does Packwerk come from?

Shopify developed it for their large monolithic system and released it in September 2020.



Packwerk is a linter for architectural boundaries.

Three core concepts

- **Package** — Directory with `package.yml`, defines a (business) unit `packs/billing`, `packs/users`, ...
- **Privacy** — Classes can be `private`: only the owning package may use them.
- **Dependency** — A package explicitly declares which other packages it is allowed to use.

The tool

- Static analysis checker
 - reads code, reports violations
 - no runtime magic, no framework lock-in

Packwerk does not tie the code more tightly to Rails — nor to Packwerk itself.

- `package_todo.yml`
 - Existing violations can be documented rather than fixed immediately — a pragmatic entry point in legacy projects.

Comparison overview

Approach	Advantage	Disadvantage
Rails monolith	Fast	Entropy
Modulith	Lightweight boundary enforcement	Design of packages
Rails Engines	Stronger isolation	Heavyweight
Microservices	Clear boundaries	Operational hell, Communication overhead



Usage at webit!

How do we use Packwerk at webit!?

JACK — A real-world example

JACK is a platform for planning and delivering food for large-scale kitchens.

- Packwerk in use **from day one**
- **23 packages** in total
- 5 domain packages: `order_management`, `portioning`, `commissioning`, `material_tracking`, `logistics`
- each with sub-packages: `domain`, `infrastructure`, `application`, `core_extension`
- 3 base packages: `core`, `architecture_core`, `events`

Structure of a domain package

Packwerk enforces boundaries between domains — and between the **layers within** a domain:

```
lib/packages/portioning/  
├─ public/           ← visible to all other packages  
├─ domain/           ← own package · visible_to: [portioning]  
├─ infrastructure/   ← own package · visible_to: [portioning]  
├─ application/      ← own package · visible_to: [portioning]  
└─ core_extension/   ← shared base types for other packages
```



Packwerk doesn't fix bad architecture.
It makes bad architecture visible.

What worked well?

- Loose coupling of domain aspects through packages with a well-defined public API
- Strict architectural rules via sub-packages — domain layer free of external dependencies (DDD)
- Tasks easy to parallelize thanks to clear package boundaries
- Refactoring within a package without external impact — thanks to a stable public API

What challenges did we face?

- Finding the right package structure and size is non-trivial — wrong divisions cost time when restructuring
- Team consensus on architectural rules is necessary — lack of shared knowledge creates friction
- UI aggregation across multiple packages more involved than in a classic monolith

Conclusion

When is Packwerk worth using?

When is Packwerk worth it?

Yes, when:

- Long-term life of a Rails monolith, especially when multiple teams work on it
- When architectural smells appear, e.g. via [Rubrowser](#)
- When services are a long-term goal but the timing hasn't come yet

Rather not, when:

- Projects in early phase — find the product first, then add structure

AI writes code faster than humans can review and verify it in depth.

Well-structured code makes that easier. Packwerk enforces that structure.

Thank you for your attention.

Any questions?

Sources:

GitHub: [Shopify/packwerk](https://github.com/Shopify/packwerk)

Diagram: [Packwerk docs](#)

Blog: [Shopify Engineering Blog](#)